

Distributed Systems: Naming



Today's lecture

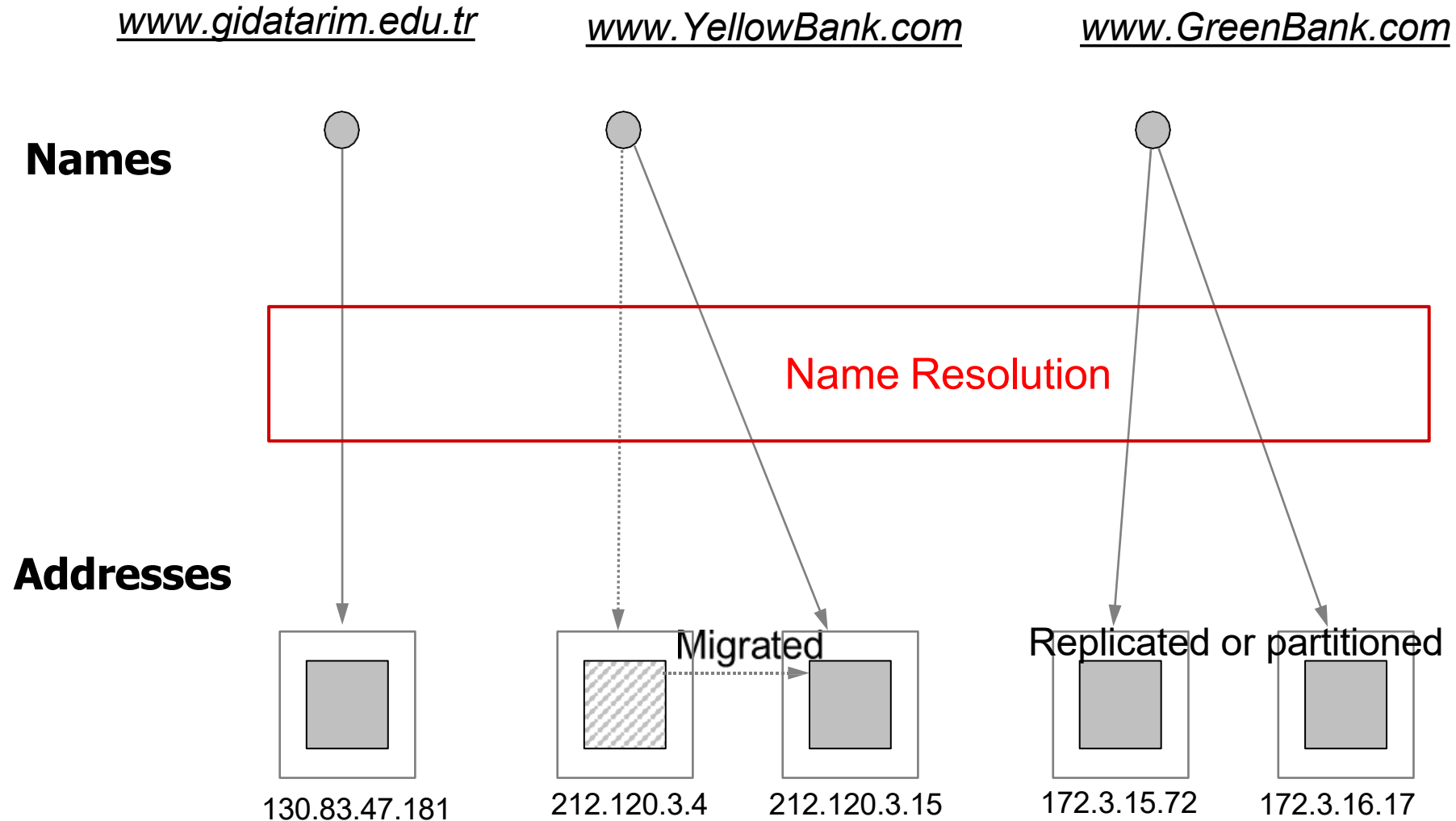
- › Names and Addresses
 - Objectives of Names
 - Terminology
 - Types of Names
- › Organization of Name Systems
 - In general
 - Flat Naming
 - Structured Naming
 - Attribute-based Naming
 - Case Study: Lightweight Directory Access Protocol (LDAP)

Names and Addresses

- **Address contains the entity's (current) location or access point.**
 - Changes whenever the entity moves to another location.
 - **Examples:** 129.69.211.1:80; 129.69.211.1:files/programs/xyz.exe
- **Name identifies the entity, independent of the entity's (current) location.**
 - Remains unchanged even if the entity changes its location.
 - **Examples:** www.google.com; files/programs/xyz.exe

Names are a prerequisite for distribution transparency!

Example



Objectives of Names

Naming plays an important role in ALL computer systems

Names are used to

- **uniquely identify entities**
 - entities can be: files, users, database tables, variables, communication links, etc.
- **operate on entities**
 - access to entity requires knowledge of its name
- **share entities**
 - exchange of names to allow for resource sharing
- **make systems secure**
 - define which active entities may access which passive entities (access control)
- **locate (mobile) entities**, etc.

Terminology

> **Alphabet**

- Set of characters
 - Example: {A, B, C}

> **Name**

- Sequence of characters of an alphabet
- Generally limited in length
 - Example: AB, AAC, ...

> **Name space**

- Set of all possible names
 - Example:
{AA, AB, AC, BA, BB, BC, CA, CB, CC}
for the alphabet {A, B, C} with a length of 2 for identifiers

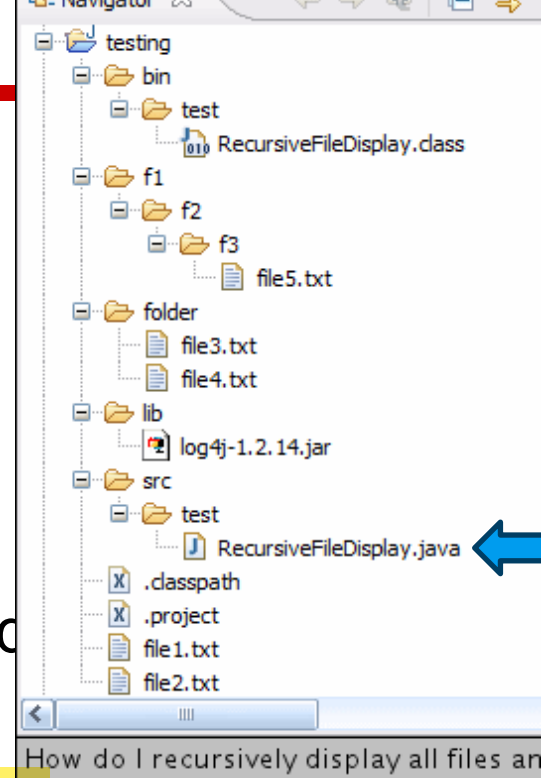
Types of Names

Flat names

- > have no structure, often random bit strings
- > contain no information whatsoever on how to locate the entity it refers to
- > good for machines rather than humans

Structured names

- > hierarchically structured name space
- > composed from human-readable (relative) names
- > Examples: Unix file names, DNS names, URLs
- > Name: testing/src/test/RecursiveFileDisplay.java



Organization of Name Systems – in General

> **Name system** maintains **name-to-address binding**

- RESOLVE name to set of addresses,
- add/remove bindings.

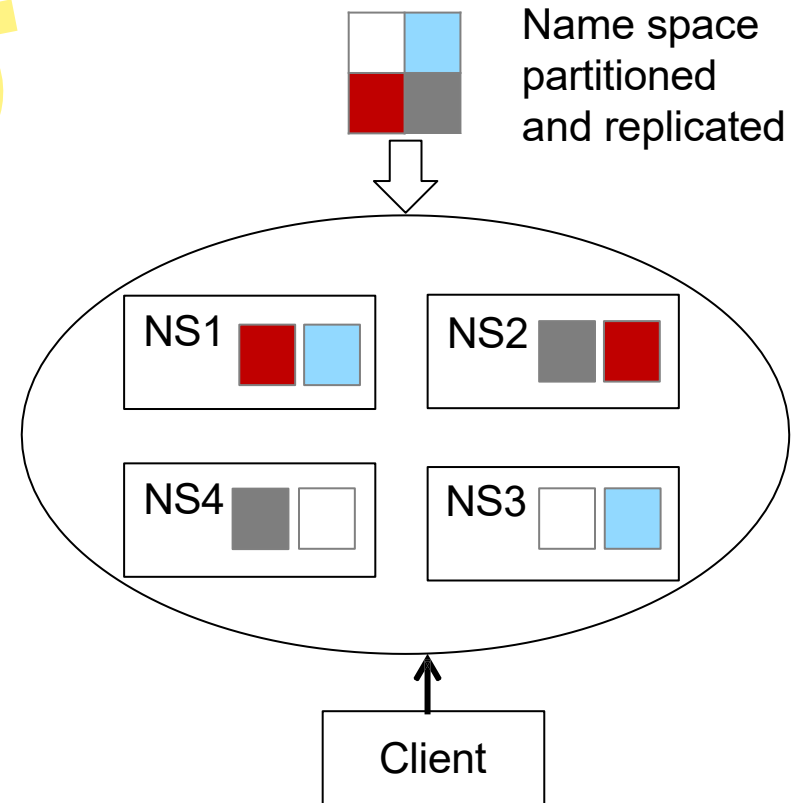
bir ismi alıp (RESOLVE) onun adresini vermektir

> **In its simplest form:**

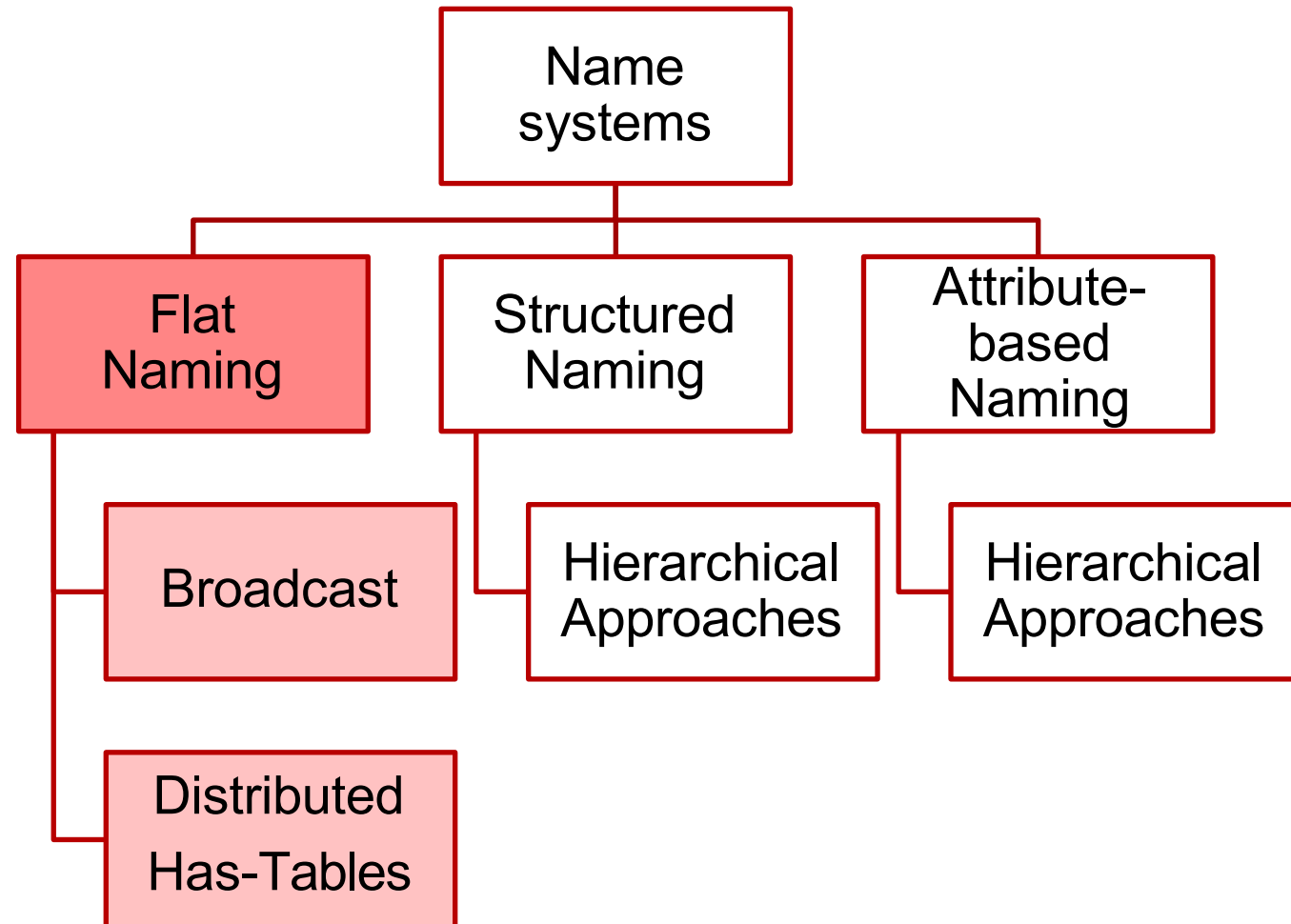
- the binding is implemented by a table of (**name, address**) pairs.

> **Name space:**

- typically **partitioned** and/or **replicated**.
- organization has strong impact on efficiency, scalability and availability.



Structured Naming



Simple Solution – Broadcast

Prerequisite: Network offers an efficient broadcast service

- Networks based on broadcast medium (e.g., LANs).

Principle of locating an entity:

- Request including name of entity is broadcast to each node.
- Each node checks whether it hosts that entity.
- Nodes that host that entity send a reply containing the entity's address.
- **Partitioned name space**, where each node only knows its local entities.

Example:

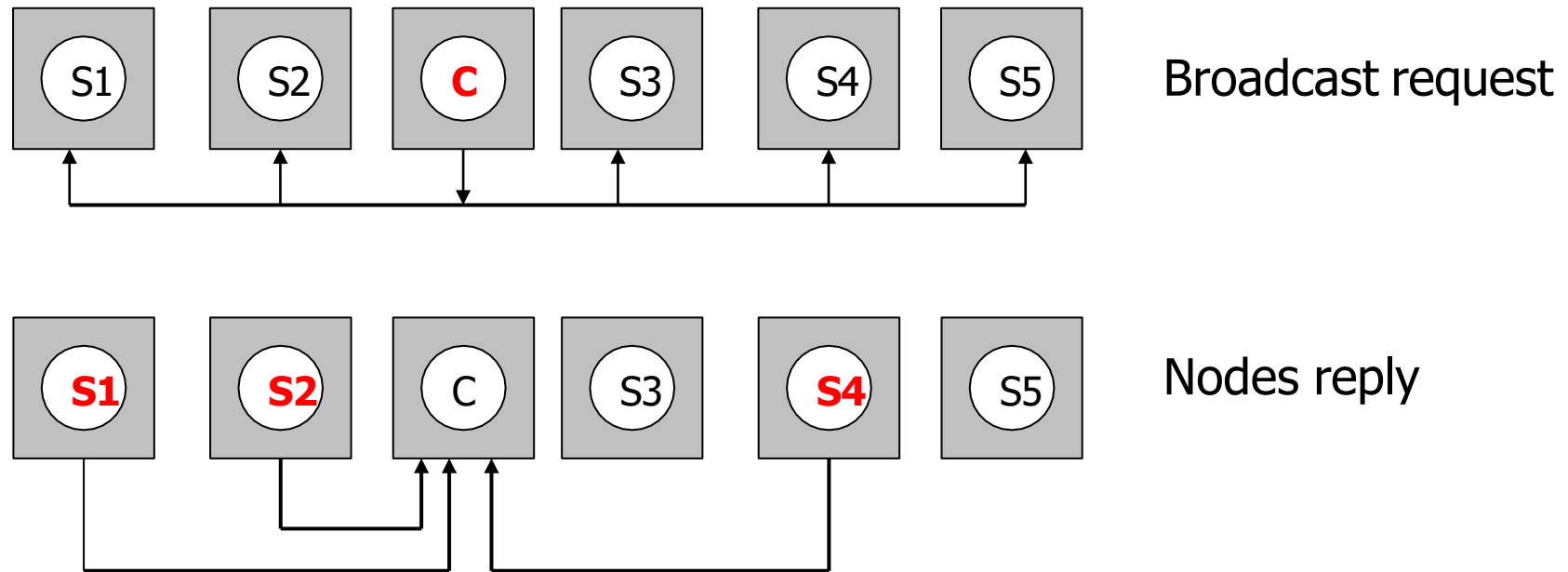
Dynamic Host Configuration Protocol – DHCP

Address Resolution Protocol - ARP

- Map IP addresses to MAC addresses.

Simple Solution –Multicast/Broadcast

Example:



Assumption: There exist 3 instances of the named object

Simple Solution – Broadcast

> Discussion:

- Name space partitioned depending on location of entities.
- Every node acts as name server (no extra name server infrastructure needed).
- Lookup is simple but not scalable in general!
 - appropriate for LANs of limited size, see DHCP

> Alternative Approach: Distributed Hash Table (DHT) – Peer to Peer approach.

- Name space partitioned according to a hash function.
- Every node can act as name server.
 - In principle, no extra name server infrastructure needed.
- Load is distributed uniformly in a simple manner.
- Scalable approach.

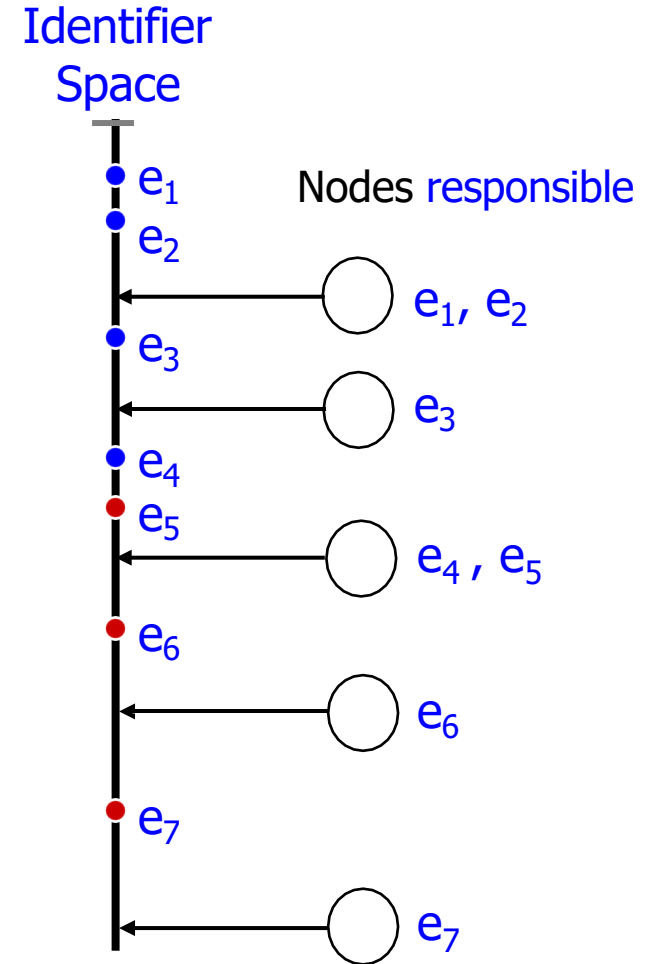
Distributed Hash Table - DHT

Basic Idea:

is in address

- > Entities are represented as (**key**, **value**)-pairs.
 - Naming: **key** corresponds to name and **value** to address.
- > **Hash function** maps **key** to **identifier space**.
- > Nodes are also mapped into identifier space and are responsible for id ranges.
 - **New node joins**: id range of neighbor is subdivided.
 - **Node leaves**: its id range is merged with a neighbor's range.

There exist many DHT-based systems, e.g., the **Chord system**.

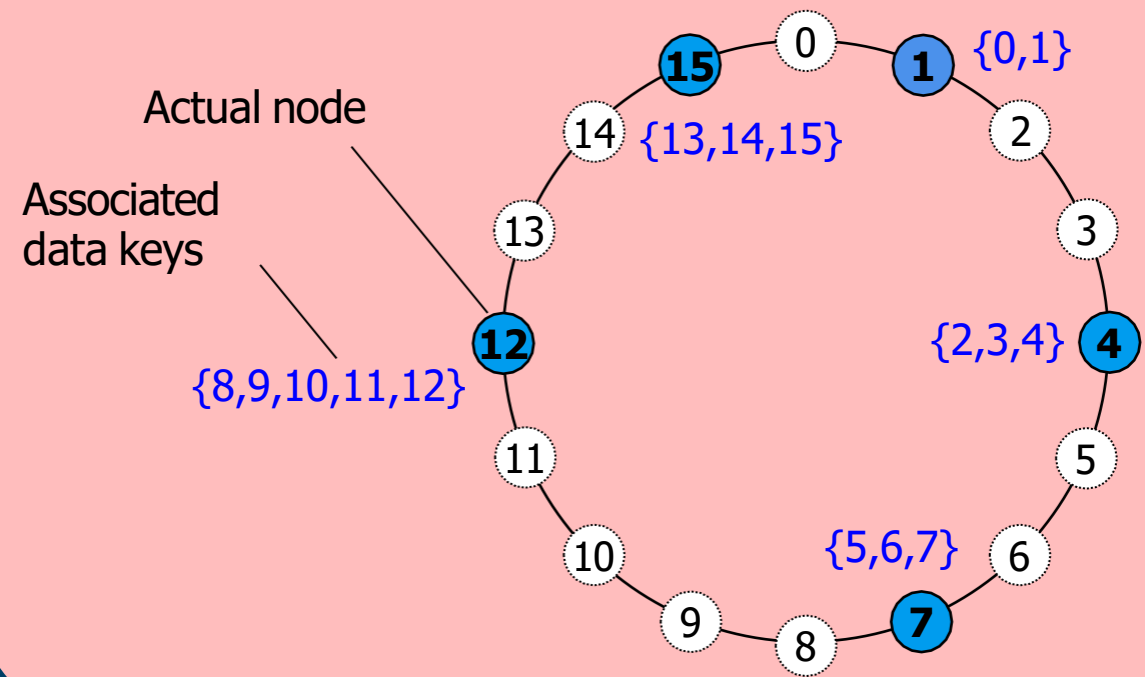


Distributed Hash Tables

Nodes are organized in a **logical ring**:

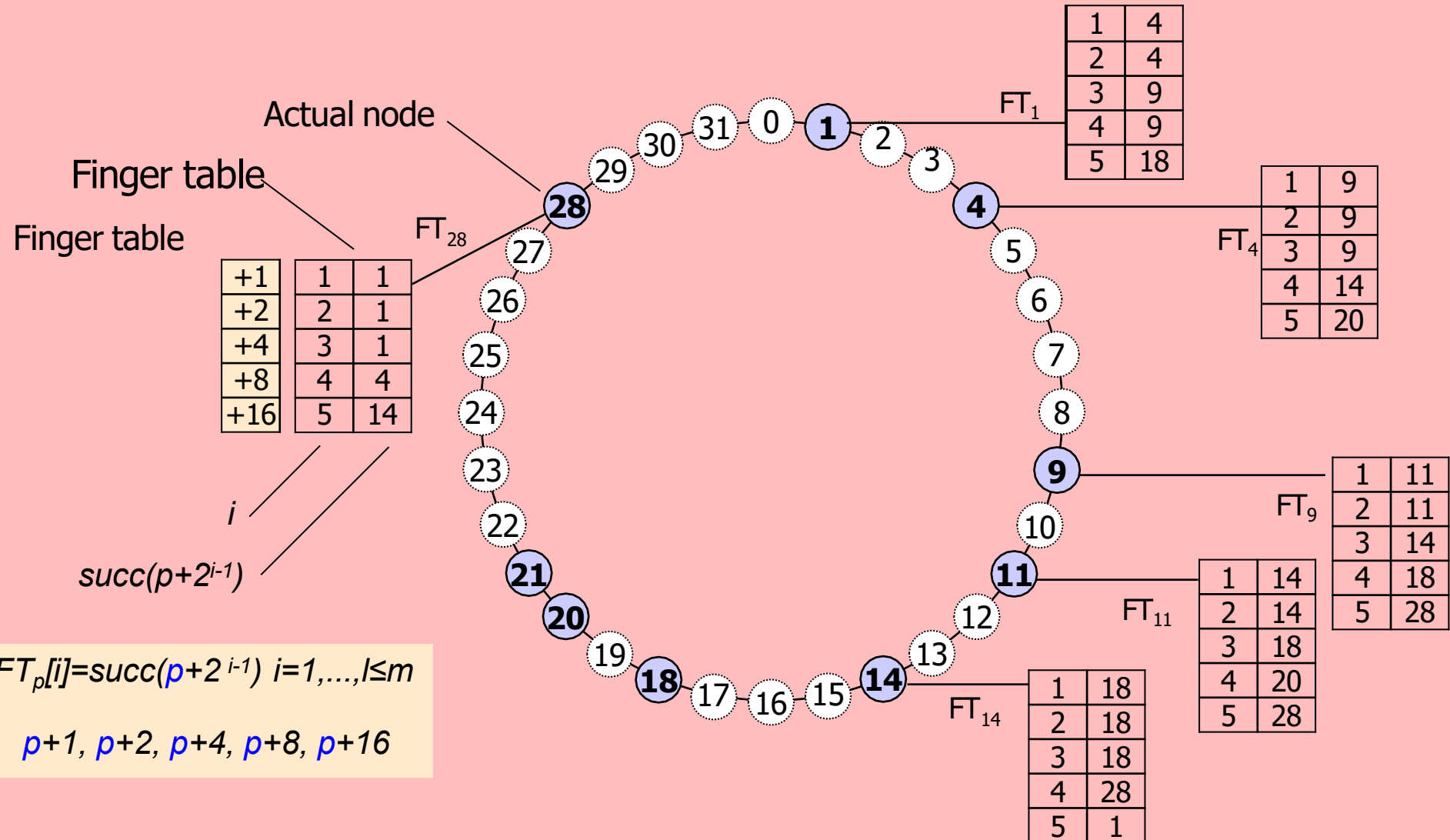
- › Ring “covers” identifier space, (i.e., circular id space).
- › **Nodes uniformly distributed** over the identifier space **due to hashing**.
- › Node p associated with each entity e with $\text{succ}(e) = p$.

Example:



Reduced Lookup time with Finger Table

Example:



Distributed Hash Tables

Lookup e=12
from node 28

Example:

Finger table

Actual node

i
 $\text{succ}(p+2^{i-1})$

1	1
2	1
3	1
4	4
5	14

1	4
2	4
3	9
4	9
5	18

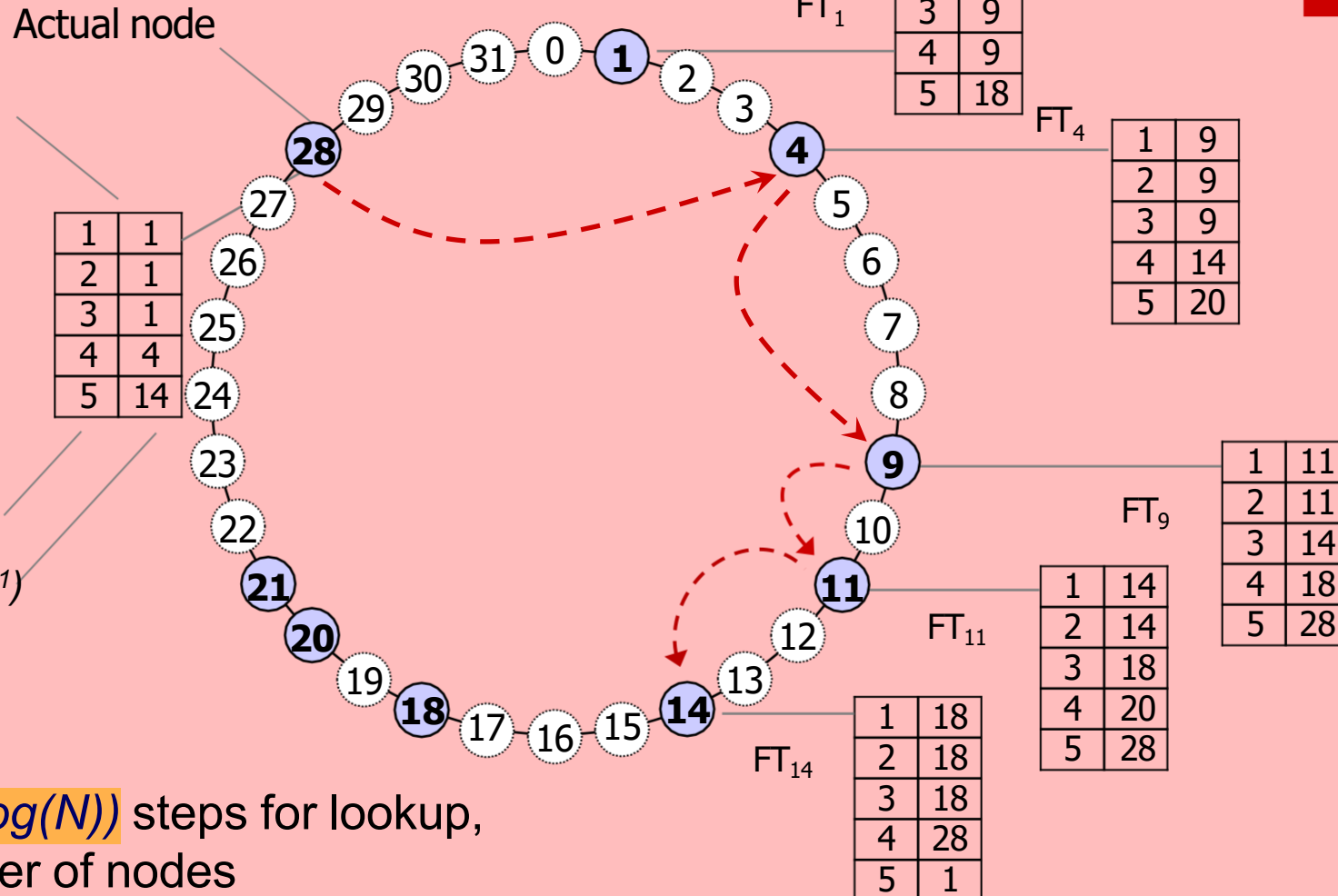
1	9
2	9
3	9
4	14
5	20

1	11
2	11
3	14
4	18
5	28

1	14
2	14
3	18
4	20
5	28

1	18
2	18
3	18
4	28
5	1

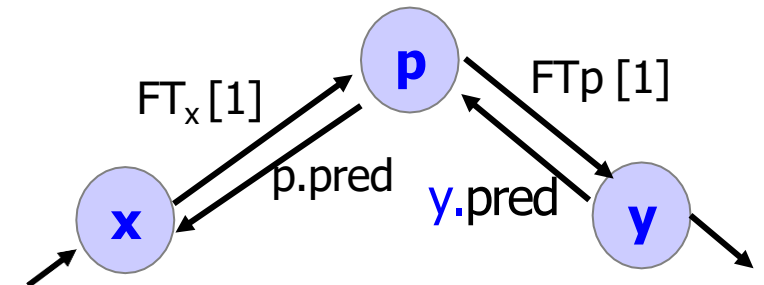
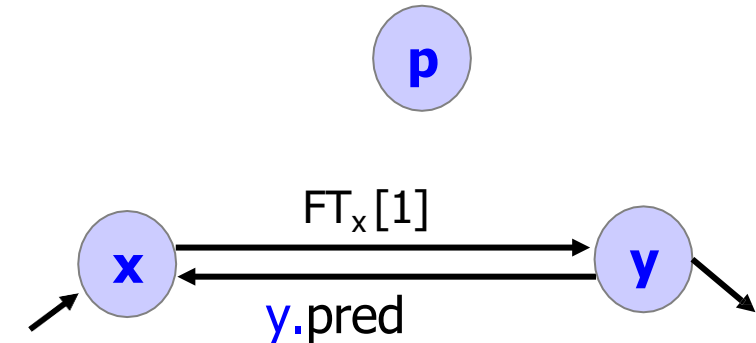
It can be shown: $O(\log(N))$ steps for lookup,
where N is the number of nodes



Distributed Hash Tables

Predecessor link is used to keep the ring **consistent**!

- > New node p joining ring:
 - p contacts arbitrary node to lookup $\text{succ}(p+1)=y$
 - Include p in ring
 - Let x be an alias of $y.\text{pred}$;
 - $\text{FT}_p[1] := y$; $p.\text{pred} := x$;
 - $\text{FT}_x[1] := p$; $y.\text{pred} := p$;
 - Initialize rest of p 's finger table, with lookups
 - Subdivide y 's entity range
- > **Fault tolerance?**
- > **Node leaving the ring is as simple.**



Distributed Hash Tables

> Discussion:

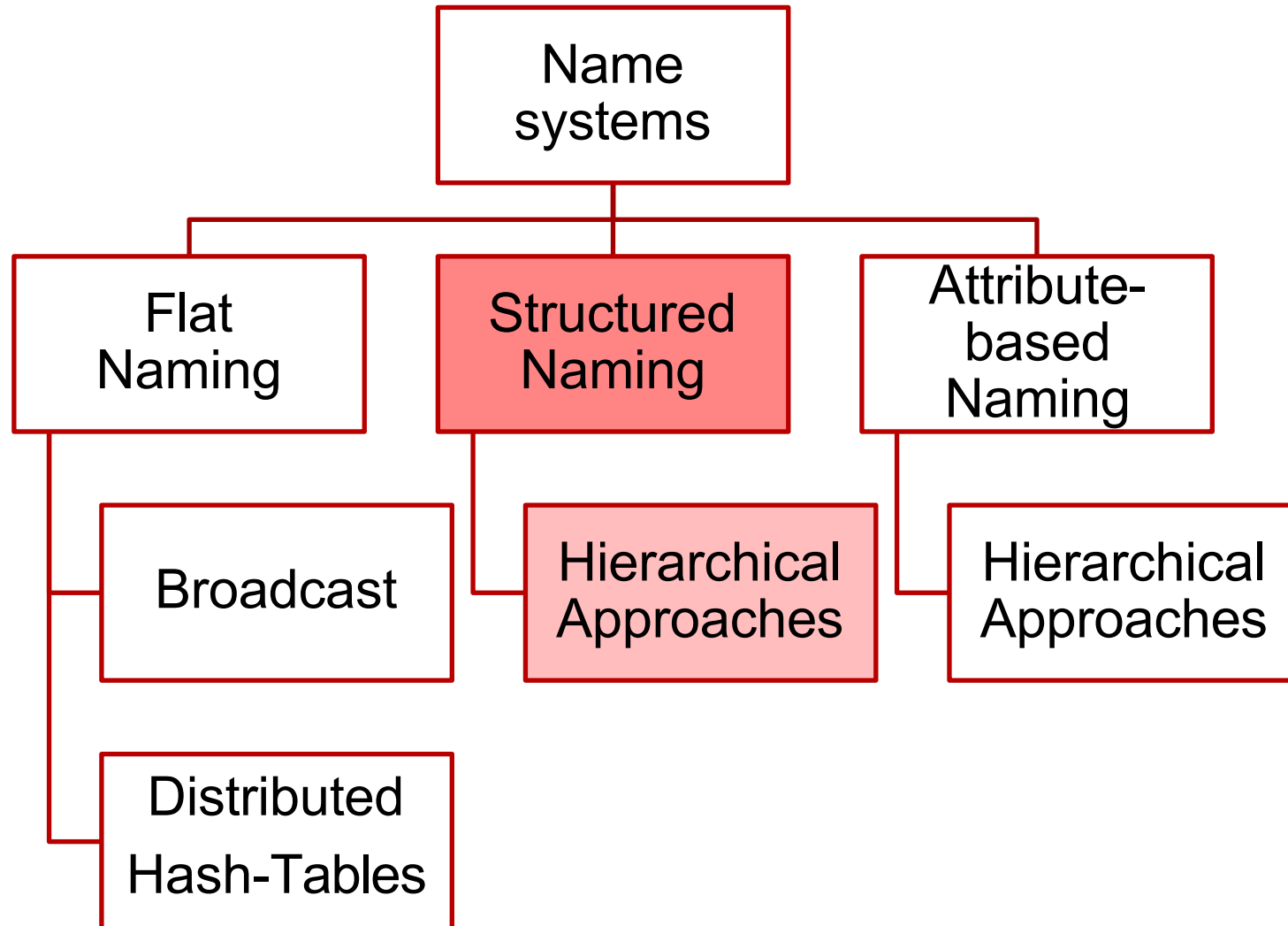
- Highly scalable.
- Uniform, self-organizing data distribution.
- Replication possible:
 - Store e additionally on $r-1$ successors of $\text{succ}(e)$, or
 - generate multiple r hash values: $H(\text{key}||c1)$; $H(\text{key}||c2)$; ... $H(\text{key}||cr)$
- Actually, it also works for structured names!
However, structure of name would not be exploited for routing.

> But: Network topology not taken into account. ağ topolojisini dikkate almaz

- Request may be routed erratically across the Internet.

Topology-based approaches (for more details see [Tanenbaum, Steen]).

Structured Naming



Structured Naming

> Structured names:

- Good for humans to use (in contrast to flat names).
- Composed of simple, human-readable names.

Example:

DNS host name: orgel.ipvs.uni-stuttgart.de

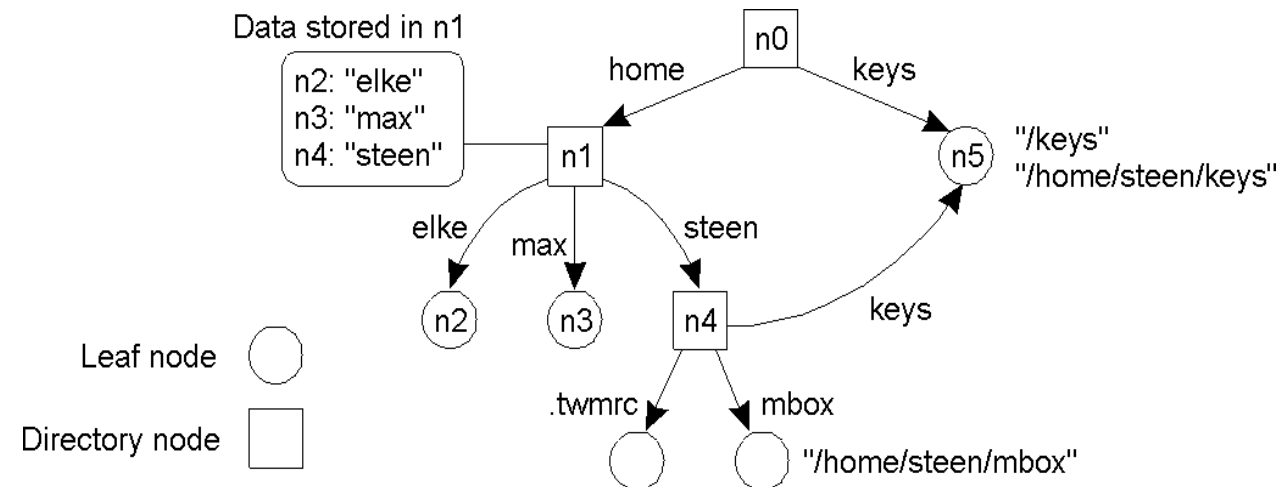
Unix file name: /users/koldehofe/lectures/distributedSystems/foils

URL: <https://www.ceng.metu.tr/ds/>

Hierarchical Name Spaces

Represented as a labeled, directed graph:

- > Leaf node represents a named entity.
 - Generally, stores information on represented entity, e.g., its address.
- > Directory node has a number of outgoing edges.
 - Stores table, includes pair (edge label, node identifier) for each outgoing edge.
- > Root node: In principle, more than one root possible (replicated).



Name Space Implementation

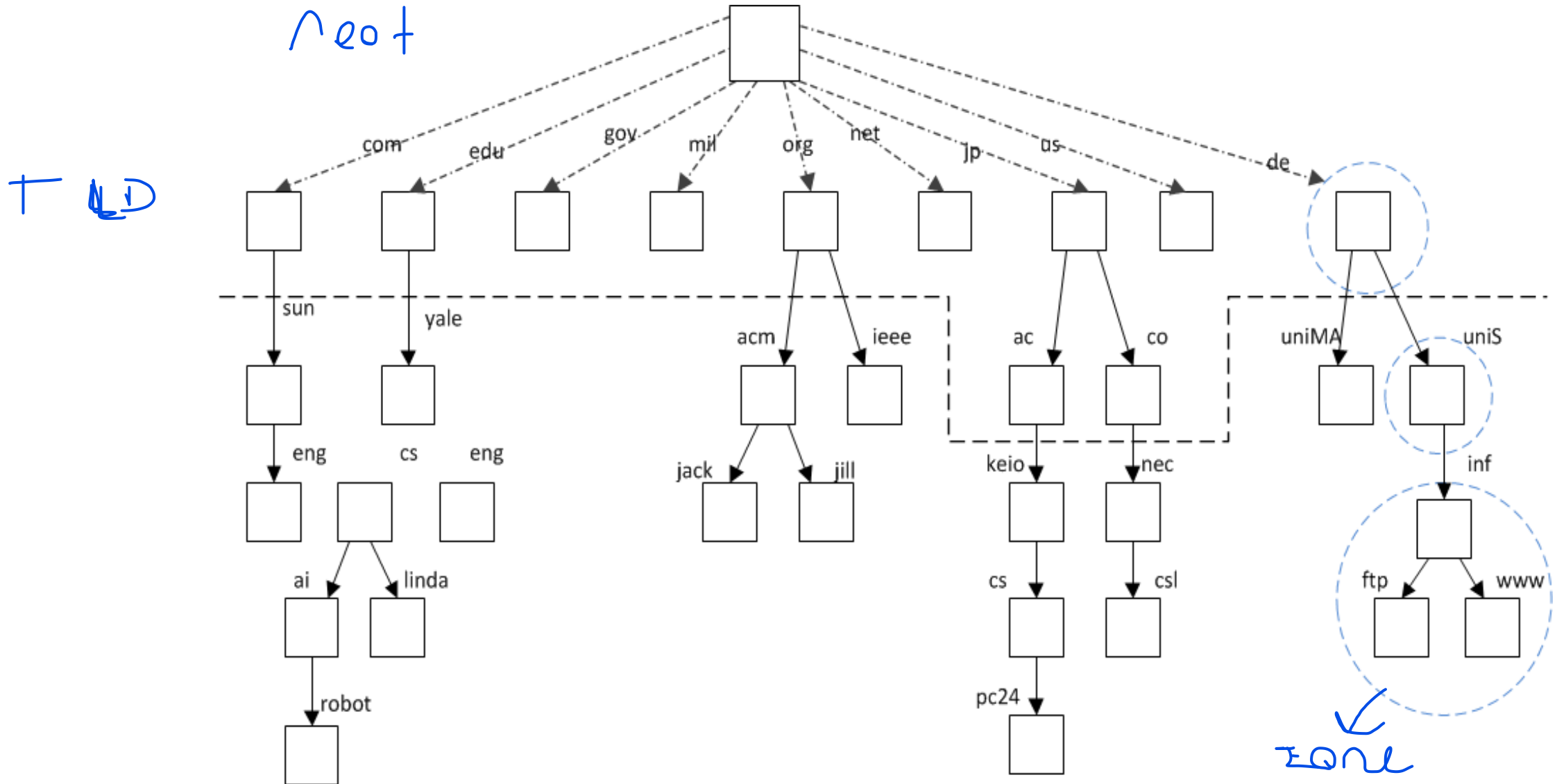
In large-scale distributed systems name space is distributed among multiple name servers

- For example, in DNS name space is partitioned into disjoint zones.
- Each zone is entirely managed by a name server.
- Zones may be replicated on multiple name servers.

Replication with Weak Consistency:

- Copies eventually converge to the same state.
- However, short-time inconsistencies possible.
- Higher availability compared to strong consistency!

Name Space Implementation



Name Space Implementation

High level name servers:

- Global relevance of naming information:
 - High availability: Replicate zones on multiple servers (weak consistency).
 - High performance: Multiple servers, client-side caching enabled.
- Naming information is quite stable (few updates):
 - Client-side caching pays off, most lookup results can be retrieved from clients' cache.
 - Hence, response times not that critical.

Low level name servers:

- local relevance of naming information.
 - Often availability not that critical.
- Naming information typically changes more frequently.
 - Hence, often caching does not work (i.e., lookup response times are critical).

Implementation of Name Resolution

In distributed naming systems resolution generally involves a number of name servers.

- › In the following we assume:
 - no replication and no **client-side caching**,
 - each client has access to **local name resolver**,
 - name to be resolved: root:<de, uniS, inf, ftp>,
 - in URL notation: ftp://ftp.inf.uniS.de

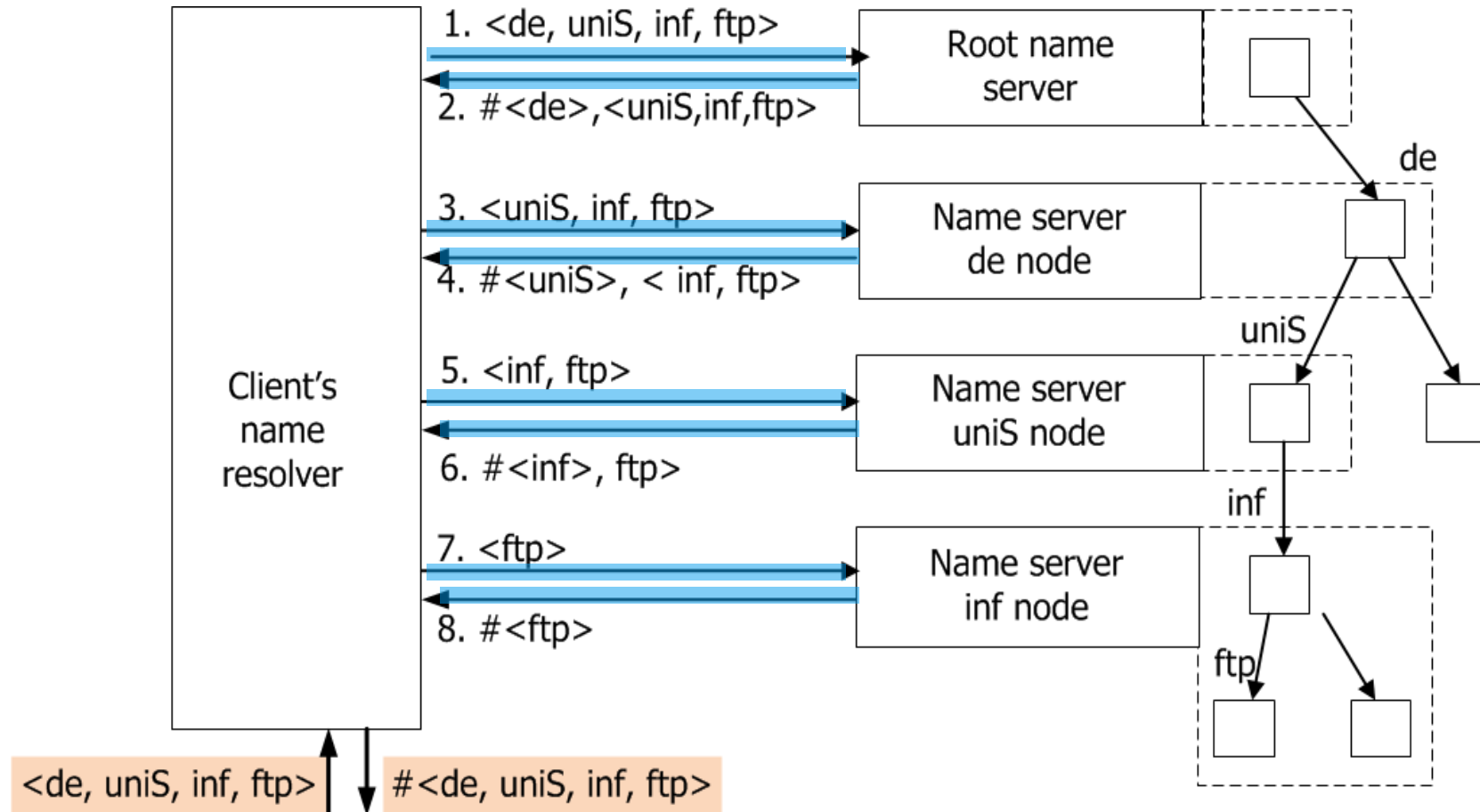
Two ways of implementing resolution:

- **iterative name resolution**,
- **recursive name resolution**. caching bayaaaa etkili

Implementation of Name Resolution

Iterative implementation:

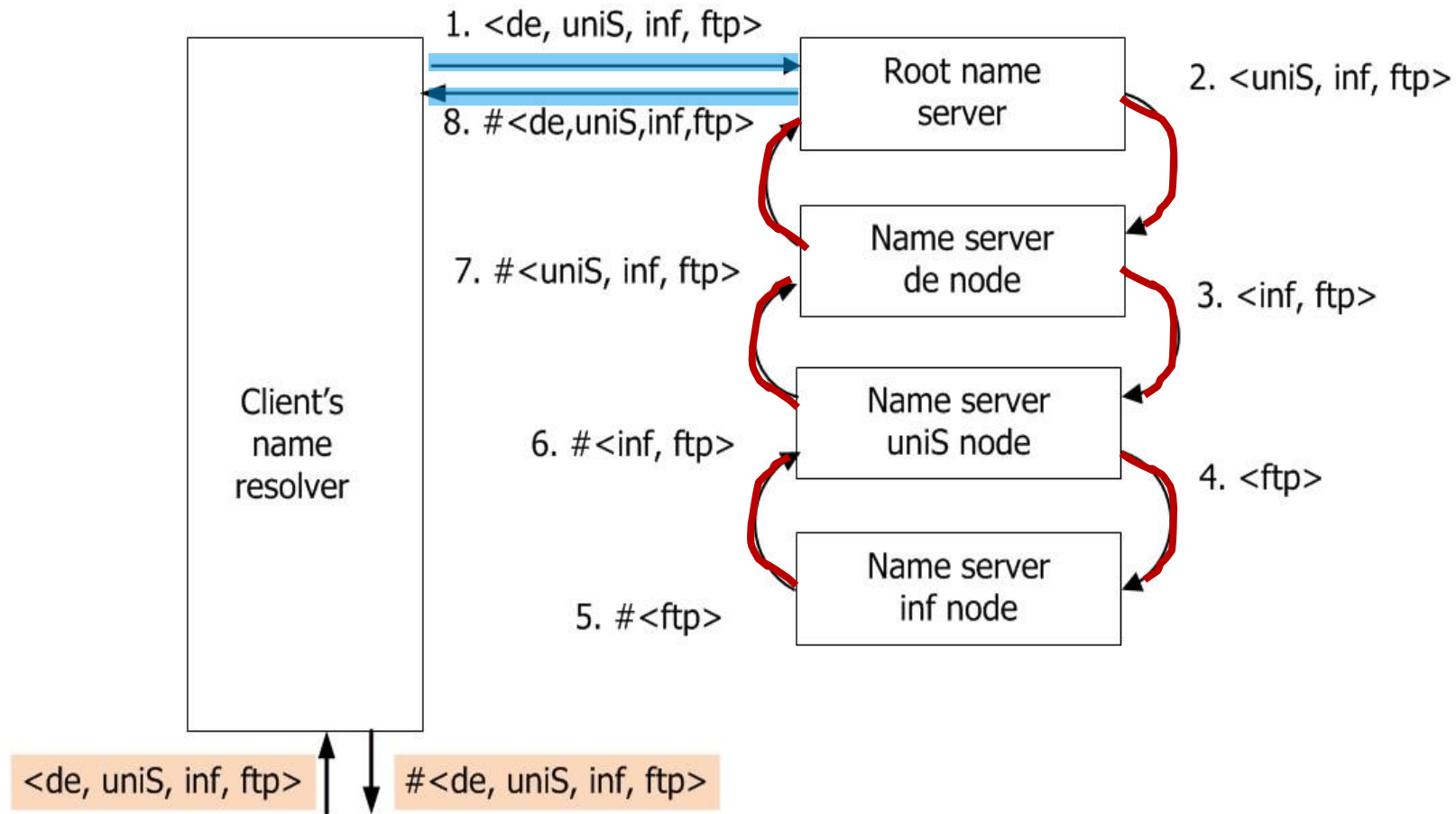
İstemci, hiyerarşideki her sunucuyla tek tek kendisi konu



Implementation of Name Resolution

Recursive implementation: istemci tüm işi ilk sorduğu sunucuya (örn: Root sunucu) devreder

1 istek
1 cevap



Implementation of Name Resolution

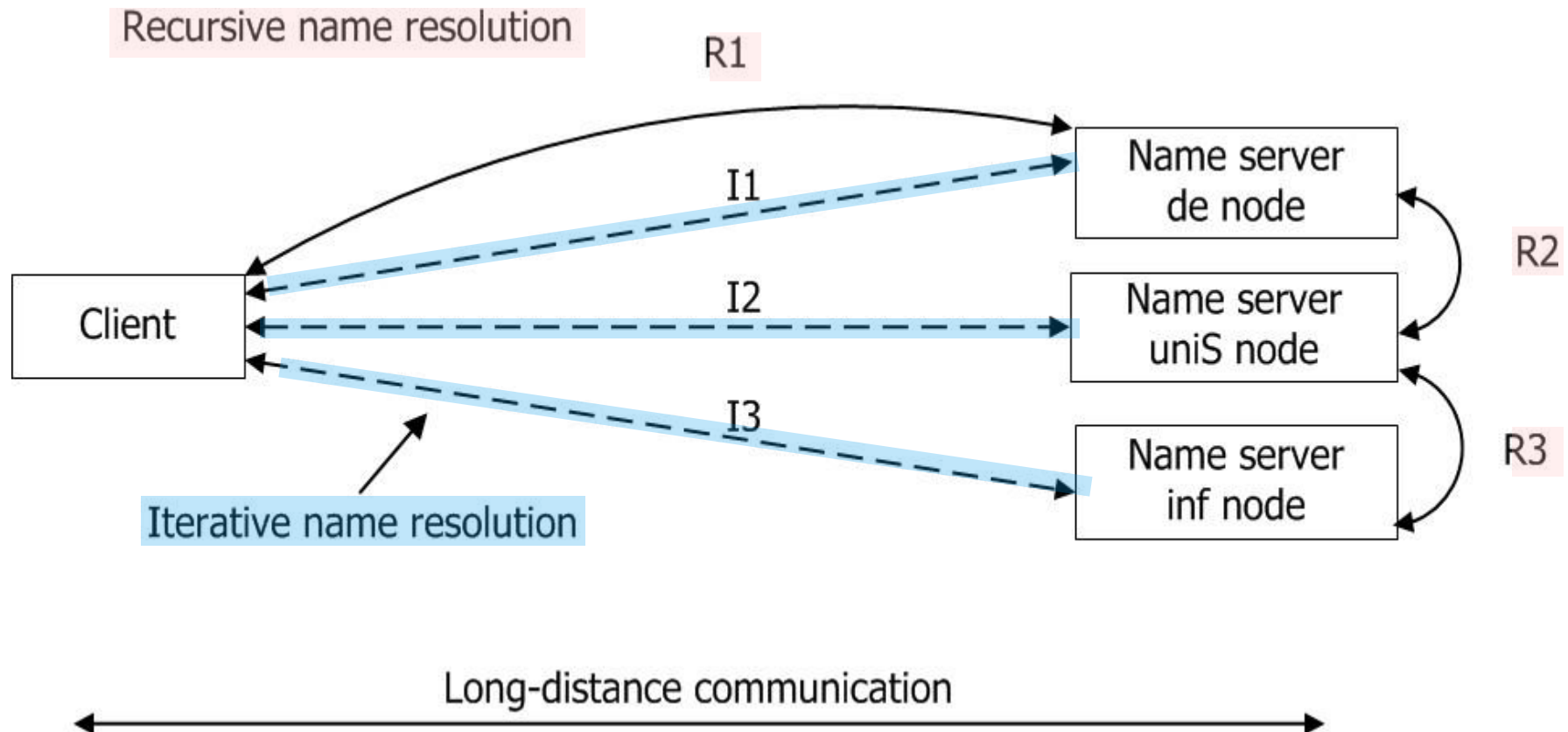
Recursive name resolution puts **higher performance** demand on **each name server**.

But:

- **Caching** becomes more effective with recursive name resolution.
 - Name servers can cache information (see following example).
 - Name server-side cache information available for all requests, (i.e., all clients can profit from cached information).
 - In contrast: content of client-cache is not shared between clients!
- **Communication** cost may be reduced (see following example).

Implementation of Name Resolution

Example: Communication cost



Hierarchical Name Spaces

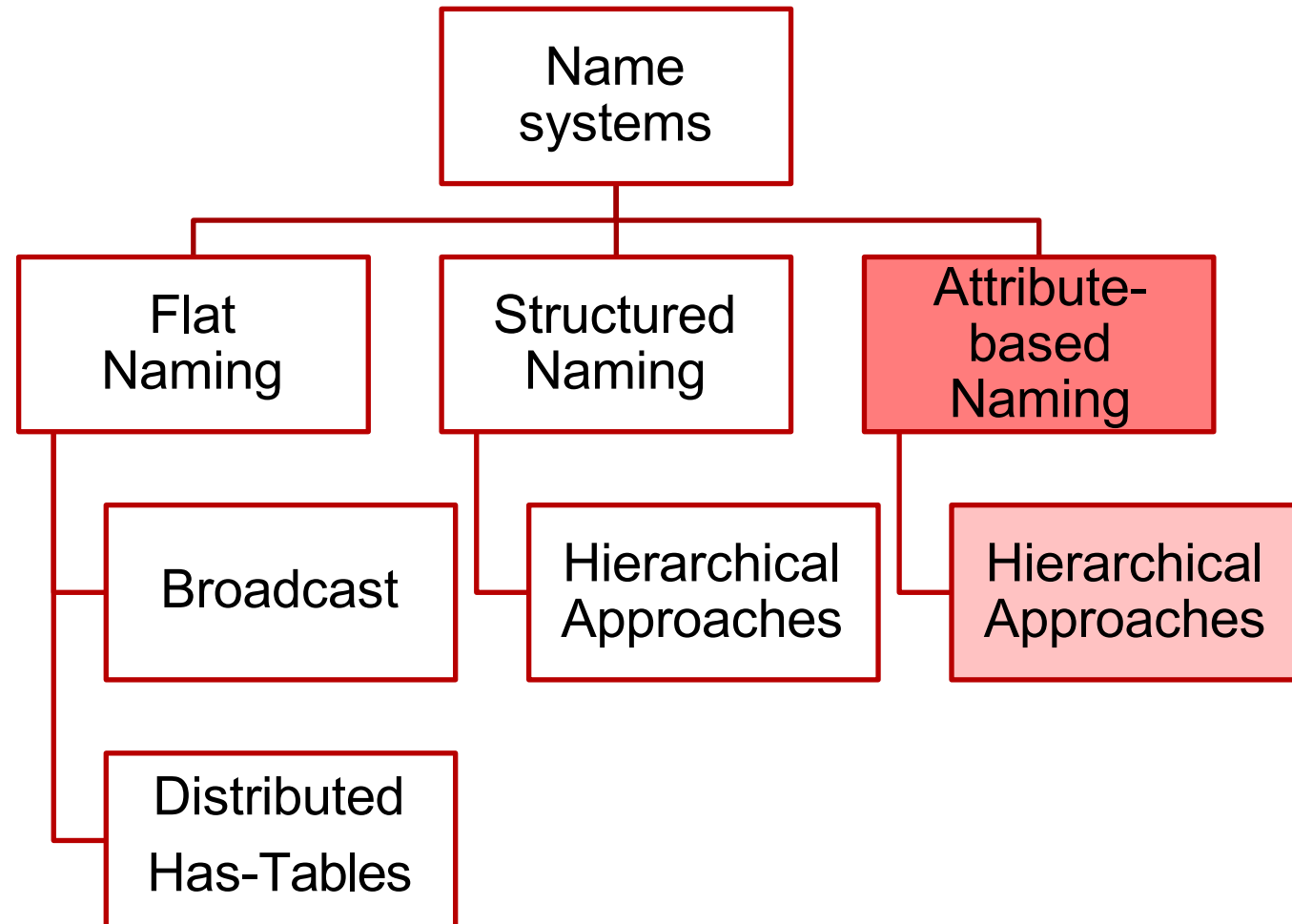
Discussion:

- › Hierarchical name space is typically partitioned, where each name server stores one partition entirely.
- › Partitions can be replicated on multiple name servers.
- › Highly scalable due to hierarchical implementation.

But:

- › Resolution may be expensive.
 - Efficiency can be improved by caching!

Structured Naming



Attribute-based Naming - Basics

So far

- › Flat and structured names: location-independent way to refer to entities.
- › Structured names: human-friendly way to name entities.
 - Entity is identified by name
 - Search can use names and possibly a predefined set of attributes (DNS)

However, sometimes users want to search for entities with certain application- or entity-specific properties.

- › Users *describe* entities rather than name them.
- › **Example:** Search(&(C=TR)(O=KGTU)(OU=MMF)(Role=SysAdmin))

Often entities are described by (attribute, value) pairs,

- › called attribute-based naming.

Example: LDAP Information Model

LDAP directory consist of a number of entries.

- Each entry consists of a collection of (attribute, value) pairs.
- Each attribute has a type.
 - Standardized types
 - (e.g., Boolean, String, Integer, Time, telephone number,...).
 - Can be extended.
- Each entry has a **DISTINGUISHED NAME**
 - identifies entry uniquely in directory
 - Any sequence of attributes can be selected

Naming conventions:

- Use of attributes:
Country, Locality, Organization, OrganizationalUnit,
CommonName (often used as unique Id for the given scope)

LDAP Information Model

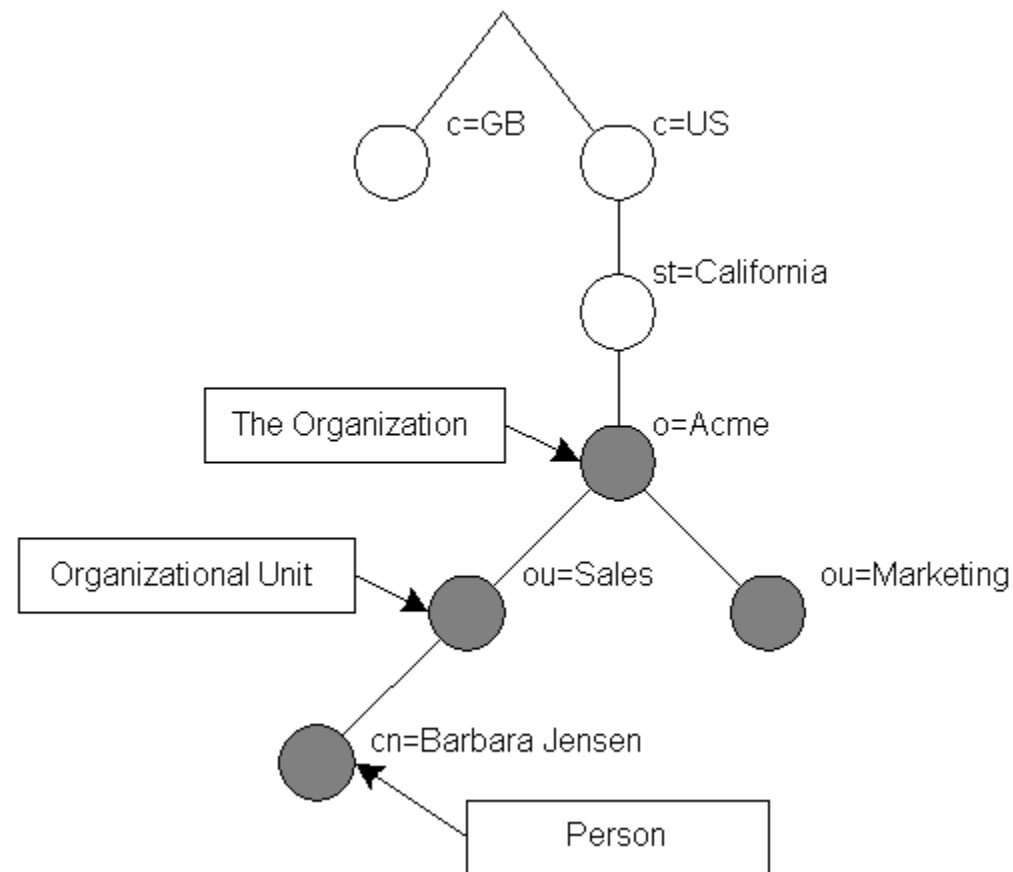
Example of an entity:

Attribute	Abbr.	Value
Country	C	TR
Locality	L	KONYA
Organization	O	KGTU
OrganizationalUnit	OU	ENG
Institute	I	CMP
Name	N	SCE
First Name	--	SomeOne
Phone	--	+90 533638888
Email	--	<u>someone@gmail.com</u>

Distinguished Name: C=TR; O=KGTU; OU=ENG; I=CMP; N=SCE

LDAP Information Model

Hierarchically organized name space: Directory Information Tree



LDAP Operations

> LDAP operations:

- add, delete, modify
- search
- ...

> Search supports:

- **Scoping**: Distinguished Name identifies sub-hierarchy of DIT to search in.
- **Filtering**: Constraints defined on attributes.
- Example for scoping and filtering:
- result = search (“&(C=TR)(O=KGTU)(!(|(N=D*)(N=S*)))”)

Scope

Filter

Discussion: Attribute-based Naming

Same characteristics as structured naming:

- Hierarchical name space is typically partitioned, where each name server stores one partition entirely.
- Partitions can be replicated on multiple name servers.
- Highly scalable due to hierarchical implementation.

Plus, powerful data model and search operations.

But:

- Search operations may be very expensive,
 - may involve a number name servers (agents) searching their local database.

Resources

- › G. F. Coulouris, J. Dillimore, T. Kindberg:
Distributed Systems – Concepts and Design, 5th Edition, Addison-Wesley, 2012. Chapter 13
- › A. S. Tanenbaum, M. van Steen:
Distributed Systems – Principles and Paradigms, 2nd Edition, Prentice Hall, 2007. Chapter 5

Slides: courtesy of **Prof. Dr. Boris Koldehofe** who developed the previous version of the slides, adapted from similar courses offered at the University of Stuttgart (**Prof. Dr. Kurt Rothermel**), and EPFL (**Prof. Dr. Rachid Guerraoui**).

